

LEARNING RESOURCE GUIDE

ISC2 CCSP: Certified Cloud Security Professional (Aug 2026) | Professional Participant Workbook

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- By the end of this lecture you will be able to identify the highest-yield topic areas within Domain 4 and allocate exam preparation effort accordingly.
- By the end of this lecture you will be able to map security activities to the correct phase of the Secure Software Development Life Cycle and explain why the exam consistently tests whether security is applied earlier rather than later.
- By the end of this lecture you will be able to distinguish between OWASP vulnerability categories and security controls, and correctly select the control that addresses the specific attack surface described in a scenario question.
- By the end of this lecture you will be able to apply the 'most appropriate for the specific context' test when four answer choices all represent valid security controls, using lifecycle phase, threat type, and attack surface as discriminating signals.
- By the end of this lecture you will be able to articulate the central principle of Domain 4 that the exam tests most consistently: security built into an application is more effective and more economical than security bolted on after the fact.

Introduction

Domain 4 accounts for sixteen percent of the CCSP exam, making it the second-largest domain in the certification. Its questions test your ability to apply secure development thinking across the full application lifecycle, from requirements through deployment and into runtime security. This is not a domain where memorising definitions produces passing results. The exam presents scenario-based questions that describe a situation and ask what a CCSP professional should recommend. You need to know not just what each concept is, but where it fits, when to use it, and what problem it is solving.

The highest-yield areas in Domain 4 are the OWASP lists and how they map to specific controls, the SSDLC phase in which each security activity belongs, threat modelling methodologies and when each is appropriate, supplemental security components matched to the correct attack surface, application architecture security trade-offs for microservices and containers, and IAM solution distinctions. Each of these areas appears repeatedly in the exam because they all test the same underlying skill: the ability to match a control to a context rather than recite a definition.

One pattern runs through virtually every Domain 4 question: the exam often presents four answers that are all real, valid security controls. The question is asking which one is most appropriate for the specific context described. The phase of the development lifecycle, the type of threat, the type of data involved, and whether the issue is a design problem or a runtime problem are all signals pointing to the right answer. Training yourself

to read those signals before looking at the answer choices is the most effective preparation strategy for this domain.

Core Concepts Explained

OWASP Lists and Control Matching

Definition: The OWASP Web Application Top 10, API Security Top 10, and LLM Top 10 appear regularly in Domain 4 exam questions. The exam tests recognition of specific vulnerability categories and the ability to match them to appropriate controls and test methodologies.

Business relevance: OWASP lists are the most widely recognised vulnerability reference frameworks in the industry. Understanding them is directly applicable to assessing and improving the security posture of cloud-hosted web and API applications, which represent the majority of the cloud application security work a practitioner will encounter.

Cloud Security Practitioner relevance: A Cloud Security Practitioner must be able to read a scenario describing an attack pattern and identify which OWASP category it represents, then identify the appropriate control. Exam questions often describe the symptom or the attack behaviour rather than naming the vulnerability category directly.

Real-world example: A CCSP exam question describes a scenario where an attacker sends specially crafted input through a web form that causes the application's database to execute unintended commands, exposing all customer records. The question asks which OWASP Web Top 10 category this represents and which control is most appropriate. A candidate who recognises this as an injection attack (OWASP A03:2021) knows the control is input validation and parameterised queries, not a WAF, not DAST, and not encryption. A candidate who knows only that 'injection is bad' without understanding the specific mechanism and the specific control will be misled by the other technically correct but context-inappropriate answer choices.

Risks or limitations: The examination trap is selecting a control that addresses a related but distinct vulnerability. For API security, knowing the distinction between Broken Object-Level Authorisation (BOLA/IDOR), where the attacker accesses another user's resources by manipulating an identifier, and Broken Function-Level Authorisation, where the attacker accesses admin functionality they should not have permission to use, is specifically tested. These two categories look similar but describe completely different attack patterns requiring different controls. Similarly for LLM applications, prompt injection (where malicious input in the user prompt manipulates the model into performing unintended actions) is distinct from model inversion or training data extraction attacks.

SSDLC Phase Mapping for Security Activities

Definition: The Secure Software Development Life Cycle (SSDLC) integrates security activities into each phase of development: security requirements during requirements gathering, threat modelling during design, SAST during the build phase, DAST during testing, and runtime monitoring during operations. The exam tests which security activity belongs in which phase.

Business relevance: Security activities applied in the correct phase are both more effective and less expensive. A vulnerability found during requirements or design costs a fraction of the effort to fix compared to one found in production. IBM's Systems Science Institute research found that fixing a defect in production costs up to fifteen times more than fixing it during requirements. This economic reality underlies the 'shift left' principle that the exam consistently rewards.

Cloud Security Practitioner relevance: A Cloud Security Practitioner must be able to identify what is wrong when a security activity is described as occurring in the wrong phase, and recommend where it should occur instead. The exam frequently describes an organisation that runs security testing only after deployment and asks what change the CCSP professional should recommend.

Real-world example: A Domain 4 exam question describes an organisation that conducts security testing only after development is complete and the application is in the testing environment. The testing frequently identifies architectural flaws that require significant rework. The question asks which change the CCSP professional should recommend first. The correct answer is to introduce threat modelling in the design phase, because the architectural flaws being found in testing could have been identified before any code was written. SAST in the build phase and DAST in testing are correct placements for those activities, but they will not catch architectural flaws. The design phase is where architectural security decisions are made, and threat modelling is the activity that systematically identifies them.

Risks or limitations: The common exam trap is selecting a security testing method that is technically valid but placed in the wrong phase, or selecting a runtime control to address a design problem. A WAF is a legitimate runtime control, but it cannot fix an insecure application design. The exam is testing whether you understand that security controls are most effective when applied to the right layer at the right time.

Threat Modelling Methodology Selection

Definition: Threat modelling methodologies covered in Domain 4 include STRIDE (mapping threats to six categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege), DREAD (a risk scoring model applied to individual threats), PASTA (a process-oriented methodology connecting technical findings to business impact), and ATASM (an attack tree approach). Each methodology is designed for a specific purpose and context.

Business relevance: Selecting the right threat modelling methodology ensures that the analysis produces results relevant to the business decision being made. STRIDE is effective for systematically identifying threat categories during design. PASTA is effective when stakeholders need to understand how technical threats translate to business impact. ATASM is effective for analysing the attack paths an adversary could use to achieve a specific objective.

Cloud Security Practitioner relevance: The exam tests whether you can identify which methodology fits a given scenario rather than reproducing scoring formulas. Read the scenario description carefully and identify the specific purpose: systematic threat enumeration by category points to STRIDE, risk scoring of individual threats points to DREAD, connecting threat findings to business risk points to PASTA, and attack path analysis points to ATASM.

Real-world example: A CCSP exam question describes a situation where a development team is in the design phase of a new cloud API and wants to systematically identify all the ways an attacker could threaten the API's security properties. The team wants to ensure no threat category is overlooked. The question asks which threat modelling methodology is most appropriate. The correct answer is STRIDE, because STRIDE provides a structured checklist of six threat categories that can be applied to each component and data flow in the system design. PASTA would be appropriate if the question described a need to connect threat findings to business impact for executive stakeholders. DREAD would be appropriate if the question described scoring and prioritising specific threats already identified.

Risks or limitations: The exam trap is selecting a methodology that sounds appropriate for threat modelling in general but does not match the specific need described. Know the distinguishing purpose of each methodology. STRIDE answers 'what can go wrong at each component'. DREAD answers 'how bad is each

specific threat'. PASTA answers 'how do technical threats connect to business risk'. ATASM answers 'what paths could an attacker follow to reach their objective'.

Supplemental Security Control Selection

Definition: Domain 4 supplemental security components include Web Application Firewalls (WAF), Database Activity Monitoring (DAM), API Gateways, and sandboxing. The exam tests the ability to match each control to the specific attack surface it is designed to cover.

Business relevance: Each supplemental control addresses a distinct layer of the application stack. Selecting the wrong control for a given attack surface adds cost without reducing risk. A practitioner who understands precisely what each control does and what it does not do is more effective in designing layered defences than one who treats these controls as interchangeable.

Cloud Security Practitioner relevance: A Cloud Security Practitioner reading a Domain 4 scenario question should identify the attack surface before looking at the answer choices. Web form used for input injection points to WAF. Privileged database user executing unusual queries points to DAM. Inconsistent access controls across multiple backend services points to API Gateway. Untrusted code that needs to be executed in isolation points to sandboxing.

Real-world example: A CCSP exam question describes a scenario where a security team has detected that a database administrator account is executing unusual queries at unusual times, accessing tables outside the normal scope of that administrator's role. The question asks which control is most appropriate to detect and alert on this behaviour in the future. The correct answer is DAM, because DAM monitors database activity at the session and query level, detecting anomalous patterns from privileged users. A WAF sits in front of web application traffic and would not observe internal database administrator queries. An API gateway manages and secures service-to-service API calls but does not monitor database query activity. Network monitoring would detect unusual traffic volumes but not the specific query content and patterns that indicate insider threat or compromised account behaviour.

Risks or limitations: The exam trap is selecting a broadly applicable control, such as WAF or network monitoring, for a scenario that describes a specific layer requiring a targeted control. Each control has a defined scope. Know what each control observes and what it cannot observe, and match the observation scope to the attack surface described in the scenario.

The Domain 4 Core Principle: Build Security In

Definition: The principle that ties Domain 4 together is that application security is not a checkpoint at the end of development. It is a discipline that runs through every phase, from threat models built during design to runtime controls monitoring behaviour in production. Security built into an application during design and development is more effective and more economical than security applied as a separate gate after the fact.

Business relevance: The 'build security in' principle has direct business implications. Applications with security requirements defined from the start have fewer post-deployment vulnerabilities requiring emergency patching, fewer data breaches requiring breach notification and remediation, and lower total cost of security over the application's lifetime. The exam consistently tests this principle because it is the practical foundation of every SSDLC framework and every cloud-native security standard.

Cloud Security Practitioner relevance: A Cloud Security Practitioner advocating for secure development practices must be able to articulate this principle in business terms. The question is not whether security should be included but at what cost and at what phase. Security included at the requirements phase is a design decision with no implementation cost. Security added as a retrofit after deployment is an emergency project with a much higher cost and lower effectiveness.

Real-world example: A CCSP exam scenario describes an organisation where the security team reviews each application after it is deployed to production and produces a list of findings for the development team to remediate. The development team treats these as low-priority because the application is already live, and most findings are addressed only when the application is next updated, which may be months later. The question asks what change the CCSP professional should recommend. The correct answer is to integrate security requirements and threat modelling into the design phase and SAST into the build pipeline, moving security from a post-deployment review to a continuous in-process activity. A WAF as a compensating control is a valid tactical option but does not address the underlying process problem. The exam is testing whether you recognise that a structural process change is needed, not just an additional runtime control.

Risks or limitations: The exam trap is recommending a runtime control or a process improvement that addresses symptoms without changing the underlying development process. When a scenario describes security problems that originate in design decisions made without security input, the most appropriate recommendation involves moving security activities into the design phase, not adding more detection controls after deployment.

Practical Applications

The following scenarios illustrate how to apply Domain 4 exam thinking to representative question patterns, demonstrating the reasoning process rather than just the answer.

Applying the Phase Signal to SSDLC Questions

A Cloud Security Practitioner preparing for the CCSP exam encounters a question describing an organisation that discovers during penetration testing that their cloud API has no authentication on several administrative endpoints, allowing any caller to invoke privileged operations. The question asks which change to the SSDLC would most effectively prevent this type of finding. The candidate identifies the signal: the issue is a missing security requirement, specifically the requirement that all administrative endpoints must enforce authentication. This is a requirements phase failure, not a testing phase failure. Running more penetration testing more frequently would find the problem earlier in future cycles but would not prevent the requirement from being missed. Threat modelling in the design phase using STRIDE would identify Elevation of Privilege and Spoofing threats against the API, leading to the authentication requirement being established before any code was written. The candidate selects threat modelling integrated into the design phase as the most effective preventive change, while recognising that DAST in the testing phase would also find the issue but represents detection rather than prevention.

Selecting the Correct Control for a Specific Attack Surface

A Cloud Security Practitioner reviewing a Domain 4 practice question reads a scenario describing an organisation that operates a cloud-hosted SaaS platform serving multiple enterprise customers. The security team has received reports that one customer's users were able to view another customer's data by modifying a numeric record identifier in an API request. The question asks which control is most appropriate to prevent this vulnerability. The candidate identifies the attack surface: this is a Broken Object-Level Authorisation (BOLA) vulnerability in the API, where the application is not verifying that the requesting user is authorised to access the specific object identified by the parameter. The candidate evaluates each option: A WAF inspects web traffic for known attack patterns but does not enforce business logic authorisation at the object level. An API gateway can enforce authentication and rate limiting but does not inherently validate per-object authorisation unless custom policies are applied. DAST would detect this vulnerability in testing. The root cause control is enforcing object-level authorisation checks in the API code itself, validating on every request that the authenticated user is

permitted to access the specific resource requested. The candidate selects this and notes that DAST should also be added to catch similar issues in future development cycles.

Distinguishing Between Valid Controls Using Context Signals

A Cloud Security Practitioner works through a Domain 4 question where all four answer choices describe legitimate security practices, none obviously wrong. The scenario describes a financial services organisation deploying a new customer-facing web application in a public cloud environment. The application handles payment card data. The development team completed the application and it is ready for production deployment. The question asks which action the CCSP professional should take first. The four options are: deploy a WAF in front of the application; conduct a threat modelling exercise; run SAST against the application source code; and perform DAST against a staging instance. The candidate identifies the context signals: the application is described as complete and ready for deployment, meaning requirements, design, and build phases are done. Threat modelling in the design phase is now past its most effective window, and SAST on complete code is valuable but represents earlier-stage activity. DAST against staging is the appropriate security activity at this phase because the application exists as a running system that can be tested before production deployment. The WAF is an operational control that should be implemented but comes after validating the application's security posture. The candidate selects DAST as the first action for the specific context described.

Best Practices

- **Read each Domain 4 scenario question to identify the phase, attack surface, and specific problem before looking at the answer choices.** Domain 4 questions are designed to test contextual judgement, not definition recall. Before reading the options, identify: What phase of the lifecycle is described? What is the specific attack surface (web application, API, database, internal service, cloud workload)? Is the problem a design issue, a build issue, a testing gap, or a runtime issue? These three signals will eliminate two or three answer choices before you evaluate the remaining options in detail.
- **For OWASP questions, know the specific mechanism and the specific prevention, not just the category name.** Exam questions on OWASP categories describe a behaviour or a symptom, not a category name. Practice reading attack descriptions and identifying the category from the behaviour. For each OWASP Web Top 10 category, know: what the attacker is doing, what application weakness enables it, and what the developer must implement to prevent it. For API Top 10, be especially clear on the BOLA versus Broken Function-Level Authorisation distinction, as these two are commonly confused in exam scenarios.
- **When a Domain 4 question asks what the CCSP professional should recommend, look for the answer that addresses the root cause, not the symptom.** Runtime controls such as WAFs, IDS, and monitoring are valuable but they address symptoms. If a scenario describes a development process problem (missing security requirements, no threat modelling, no SAST in the pipeline), the answer that addresses the root cause will involve adding the correct security activity to the correct SSDLC phase. Recommending a WAF for an injection vulnerability that should have been prevented by parameterised queries answers the symptom, not the cause. The exam rewards answers that address the structural problem.
- **For container and Kubernetes questions, keep the layered model in mind: image hygiene, cluster hardening, network policy, and runtime monitoring are four distinct layers, each with distinct controls.** Exam questions on container security often describe a problem at one specific layer and ask which control addresses it. A compromised image reaching production is an image hygiene failure, addressed by scanning and admission control. Unrestricted pod-to-pod communication is a network policy failure. A service account with excessive permissions is an RBAC failure. A container executing unusual system calls is addressed by

runtime monitoring. Do not mix layers in your answer unless the question explicitly asks for a comprehensive approach.

- **For IAM questions, apply the force-multiplier test to SSO and the factor-category test to MFA.** For any SSO scenario, ask: what protects the identity provider session? If MFA is absent or weak, SSO is a risk concentration, not a risk reduction. For any MFA scenario, ask: do the two factors come from different categories? Two knowledge factors (password plus security question) is single-factor authentication presented in two steps. A password plus a TOTP code or hardware key is genuine two-factor authentication. Apply these tests mechanically to each scenario and the correct answer becomes immediately apparent.

Common Mistakes and Misconceptions

Misconception: Selecting the most comprehensive or most sophisticated control for every Domain 4 question, regardless of what the scenario describes.

Correct approach: The correct answer is the most appropriate control for the specific context, not the most comprehensive one. A penetration test is more comprehensive than DAST but is not the correct answer for a question asking about automated security testing integrated into a CI/CD pipeline. Match the control to the specific need described in the scenario.

Misconception: Treating SSDLC phase questions as asking where testing should happen and selecting DAST or SAST for every scenario involving a security gap.

Correct approach: Testing activities find problems; design activities prevent them. When a scenario describes recurring architectural security flaws or missing security requirements, the correct answer addresses the design or requirements phase, not the testing phase. Threat modelling in design prevents the category of problem that testing would otherwise repeatedly find.

Misconception: Confusing Broken Object-Level Authorisation (BOLA) with Broken Function-Level Authorisation in API security scenarios, and selecting the same control for both.

Correct approach: BOLA involves accessing a specific resource that belongs to another user by manipulating an identifier. The control is per-object authorisation checks in the API logic. Broken Function-Level Authorisation involves accessing admin-level API functions without appropriate permission. The control is enforcing function-level permission checks. These require different code-level fixes, and the exam distinguishes between them in scenario descriptions.

Misconception: Reading 'build security in' as recommending that security teams replace runtime controls with development process improvements.

Correct approach: The build-security-in principle calls for integrating security into the development process from the start, not eliminating runtime controls. The point is that both are needed: security requirements, threat modelling, and SAST prevent vulnerabilities from being built in; WAFs, monitoring, and runtime controls detect and limit impact when vulnerabilities reach production. The principle determines which layer to prioritise when resources are constrained and when the process fails.

table: Domain 4 Exam Signal Reference

Use this table as a quick-reference during exam preparation. Match the signal in the question to the topic area and the discriminating test that identifies the correct answer.

Signal in Question Scenario	Topic Area	Discriminating Test	Common Wrong Answer
Web form, input causes database to execute unintended commands	OWASP Injection (A03)	Control = parameterised queries; not WAF (WAF is compensating, not preventive)	WAF or output encoding instead of input validation and parameterised queries
User modifies record ID in API call to access another user's record	OWASP API BOLA	Control = per-object authorisation check in API logic	Authentication enforcement (authentication is already working; authorisation at object level is the gap)
Attacker calls admin API function with a regular user token	OWASP Broken Function-Level Authorisation	Control = function-level permission check	Confused with BOLA; same recommendation but different mechanism
LLM application manipulated by crafted prompt to perform unintended actions	OWASP LLM Prompt Injection	Newest category; appears in AI cloud application scenarios	General input validation instead of prompt-specific guardrails
Recurring architectural security flaws found late in SDLC	SSDLC phase mismatch	Root cause = no threat modelling in design phase	Recommending more DAST or penetration testing (detection, not prevention)
SAST, DAST, or IAST question on correct phase placement	SSDLC testing methods	SAST = build phase, no running app needed; DAST = testing phase, live app required	SAST placed in testing phase or DAST in build phase
Systematically identify all threat categories for a new design	Threat modelling methodology	STRIDE; enumerates six categories across components and data flows	PASTA (for business impact) or DREAD (for scoring known threats)
Connect threat findings to business impact for executives	Threat modelling methodology	PASTA	STRIDE (identifies threats but does not frame business impact)
Web form used for malicious input injection	Supplemental controls	WAF	DAM (DAM monitors database queries, not web form input)
Privileged DB user executing unusual queries	Supplemental controls	DAM	WAF (WAF does not observe internal DB query patterns)
Inconsistent access controls across backend services	Supplemental controls	API Gateway	WAF (WAF is for web application layer, not inter-service API access control)
SSO deployed; question tests security posture	IAM - SSO force multiplier	Is MFA mandatory on the identity provider?	Assuming SSO improves security regardless of identity provider authentication strength
Two verification steps described; is this MFA?	IAM - MFA factor categories	Do the two factors come from different categories?	Counting steps as factors; two knowledge factors are still single-factor authentication
Employees using unapproved cloud storage	IAM - CASB	CASB for discovery and policy enforcement across sanctioned and shadow IT	Federated identity (addresses authentication for known apps, not shadow IT discovery)
Application secrets in source code or config	IAM - Secrets Management	Secrets management platform with runtime injection and rotation	Environment variables (improvement over inline code but still exposed)

How to use this: This table covers the most frequently tested Domain 4 scenario patterns. For each question on the exam, identify the signal first, then apply the discriminating test before evaluating answer choices.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

Q1. A CCSP exam question describes an organisation that discovers during production penetration testing that their cloud application has no input validation on user-supplied fields, allowing SQL injection attacks that expose the customer database. The question asks which SSDLC change would most effectively prevent this type of finding in future development cycles. Which answer demonstrates correct understanding of the 'build security in' principle?

- A) A. Deploy a WAF in front of the production application to block SQL injection attempts
- B) B. Run DAST against the application in the testing environment before each production release
- C) C. Include input validation and parameterised query requirements in the security requirements phase and enforce them through SAST in the build pipeline
- D) D. Conduct a penetration test against the application in staging before each production release

Answer: C. Include input validation and parameterised query requirements in the security requirements phase and enforce them through SAST in the build pipeline

Q2. A Domain 4 exam scenario describes a security team that wants to systematically identify all the ways an attacker could threaten a new cloud API being designed. They want to ensure no threat category is overlooked during the design phase. Which threat modelling methodology is most appropriate for this goal?

- A) A. DREAD, because it provides a numerical risk score for each identified threat
- B) B. PASTA, because it connects threat findings to business impact for stakeholder reporting
- C) C. STRIDE, because it provides a structured checklist of six threat categories that can be applied to each component and data flow
- D) D. ATASM, because it identifies attack paths an adversary could follow to reach their objective

Answer: C. STRIDE, because it provides a structured checklist of six threat categories that can be applied to each component and data flow

Q3. A CCSP exam question describes a scenario where a cloud SaaS API allows users to retrieve their own profile data by calling GET /users/{id}. An attacker discovers that changing the ID value to another user's ID returns that user's profile data without any error. Which OWASP API Security Top 10 category does this represent, and what is the appropriate control?

- A) A. Broken Function-Level Authorisation; the control is restricting access to admin-level API functions
- B) B. Broken Object-Level Authorisation; the control is verifying on every request that the authenticated user is permitted to access the specific object identified by the parameter
- C) C. Injection; the control is input validation and parameterised queries applied to the ID parameter
- D) D. Security Misconfiguration; the control is reviewing API endpoint configurations to ensure they require authentication

Answer: B. Broken Object-Level Authorisation; the control is verifying on every request that the authenticated user is permitted to access the specific object identified by the parameter

Q4. A Domain 4 question describes a container security scenario where an attacker gained code execution in a web application container and then accessed the payment processing service's database credentials stored in another pod's environment variables. Which two controls, if both had been in place, would most directly have prevented the lateral movement to the payment service?

- A) A. Container image scanning in the CI/CD pipeline and DAST against the web application before deployment
- B) B. Default-deny Kubernetes network policies restricting pod-to-pod communication and a secrets management platform replacing environment variable credential storage

- C) C. RBAC least-privilege service accounts for the web tier and Kubernetes API server audit logging
- D) D. Docker Content Trust for image signing and runtime anomaly detection using Falco

Answer: B. Default-deny Kubernetes network policies restricting pod-to-pod communication and a secrets management platform replacing environment variable credential storage

Q5. A Cloud Security Practitioner is reviewing Domain 4 practice questions and encounters a scenario where an organisation has deployed SSO across twenty applications and an employee's SSO session is used by an attacker who obtained the session token through a phishing attack. The session token gave the attacker access to all twenty applications. Which assessment most accurately describes the security situation?

- A) A. The SSO implementation is the security failure; the organisation should revert to separate passwords for each application
- B) B. The SSO implementation is correct, but it was deployed without mandatory MFA on the identity provider, which would have blocked the phishing attack from establishing a valid session
- C) C. SSO is inherently insecure for applications handling sensitive data and should not be used in regulated environments
- D) D. The phishing attack bypassed SSO security controls, which means SSO is not effective against credential compromise

Answer: B. The SSO implementation is correct, but it was deployed without mandatory MFA on the identity provider, which would have blocked the phishing attack from establishing a valid session

Reflection Questions

1. Review the SSDLC phase mapping for each security activity: requirements, design, build, testing, and operations. For each activity (threat modelling, SAST, DAST, penetration testing, runtime monitoring), can you state precisely which phase it belongs in and why? Can you explain what problem each activity finds and what it cannot find?
2. For each OWASP Web Top 10 and API Top 10 category you have studied, practice reading an attack description and identifying the category without seeing the category name. Then identify the specific control, not just the category. This is the skill the exam tests.
3. Consider a real or hypothetical cloud application you are familiar with. Work through a STRIDE analysis on one component or API endpoint. What Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege threats apply? Which ones would you prioritise based on the data the application handles?
4. When you encounter a Domain 4 practice question with four plausible answers, practice stating the phase, attack surface, and specific problem before reading the options. Does this process change which answer you select compared to reading the options first?
5. How would you explain the 'build security in' principle to a development manager who sees security requirements and threat modelling as a development speed constraint? What evidence or examples from the industry would you use to make the economic case?
6. Review the supplemental security control matrix from this domain. For each control (WAF, DAM, API Gateway, sandboxing), can you describe a scenario where that control is the correct answer and a different scenario where it would be the wrong answer even though it is a legitimate security control?

Key Takeaways

- Domain 4 accounts for sixteen percent of the CCSP exam and tests contextual application of security knowledge across the full application lifecycle, not definition recall.
- For OWASP questions, identify the category from the attack behaviour described, then select the specific prevention control, not just the category name. Know BOLA versus Broken Function-Level Authorisation distinctly.
- Map security activities to SSDLC phases before selecting an answer. Security requirements belong in requirements; threat modelling in design; SAST in build; DAST in testing; runtime monitoring in operations.
- STRIDE is for systematic threat enumeration across design components. DREAD is for scoring individual threats. PASTA connects findings to business impact. ATASM analyses specific attack paths.
- For supplemental control questions, identify the attack surface first: web form input points to WAF, privileged database user behaviour points to DAM, inter-service API access control points to API Gateway, untrusted code execution points to sandboxing.
- SSO is a force multiplier. The exam tests whether you recognise that its security value depends entirely on the strength of authentication on the identity provider, specifically whether MFA is mandatory.
- The central Domain 4 principle: application security is not a checkpoint at the end of development. Security built into an application is more effective and more economical than security bolted on after the fact. When the exam asks what to recommend first, look for the answer that addresses the root cause in the correct phase.
- When all four answer choices describe valid security controls, the discriminating signals are the lifecycle phase, the attack surface, and whether the problem is structural (design phase solution) or operational (runtime control solution).

Recommended Next Actions

Put this lecture to work within the next seven days:

7. Work through fifteen to twenty Domain 4 practice questions applying the signal-identification process: before reading the options, write down the phase, attack surface, and problem type. Review whether this process changed your answer selection accuracy.
8. Create flashcards for each OWASP Web Top 10 and API Top 10 category with three columns: the attack behaviour description, the specific vulnerability name, and the specific prevention control. Practice reading the behaviour and naming both the category and the control.
9. Review each threat modelling methodology with one distinguishing phrase: STRIDE for systematic threat category enumeration, DREAD for risk scoring, PASTA for business impact framing, ATASM for attack path analysis. Apply this to three to four practice questions on methodology selection.
10. Complete a timed Domain 4 practice set of twenty-five questions, then review each incorrect answer to identify which signal you missed: phase, attack surface, structural versus operational, or root cause versus symptom.